REKOLT Planters' Cooperative Produce Tracker Design

1. Scope statement

This project builds a console app for REKOLT Planters' Cooperative that records produce deliveries, grades them, calculates payment using a fixed set of rules, and produces one season report at the end.

The aim is to replace a paper-slip process that used to take a treasurer around eleven days to work through by hand.

Assumptions

- Delivery data lives only in memory while the program runs. It does not need to be saved to a file or reloaded between runs.
- The season report is the one exception: it must still be written to a real Word document (.docx) when requested.
- Only one cooperative and one season are handled at a time. There is no login, no multi-season history, and no multi-user access.
- The person running the program acts as the clerk and the treasurer at once. There is no separate user role in the software.

2. Requirements

Functional requirements:

- The system will let the user record a delivery by entering member ID, member name, produce code, mass, quality score, and week.
- The system will re-prompt with a clear reason whenever an entered value fails validation, and shall never crash on bad input.
- The system will grade a delivery as A, B, C, or REJECT based on the quality score, using the exact boundaries in the payment rules.
- The system will calculate net payable through the five fixed steps: base value, grade multiplier, category multiplier, commission, transport levy.
- The system will record a REJECT delivery with zero value and zero deductions, while still counting it in volume statistics.
- The system will display season figures on screen: total payment per member, a weekly volume grid, and a ranked list of deliveries.
- The system will let the user search for a member's deliveries by member ID, and shall handle the case where the member does not exist.
- The system will generate one Word document containing a payment statement section for every member, plus a season-total closing section.

Non-functional requirements

- Every monetary figure in the generated report must match manual calculation to the cent.

- The console interface must remain usable by a treasurer with no programming background, using plain prompts and error messages.
- The program must run from a clean clone with a single documented command, using only JDK 17 or later and Maven.
- Adding a new produce category in the future should require touching as few classes as possible.

3. Noun-verb analysis

Nouns from the scenario and rules became candidate classes. Verbs became candidate methods on those classes.

Candidate class	**Why it was kept**
Produce (abstract) / CerealProduce / PerishableProduce / CashCropProduce	Each crop category values a delivery differently (its own multiplier). This is a textbook case for inheritance: shared shape, different behaviour.
Delivery	The central record of the system. Every calculation, sort, search, and report line starts from a Delivery.
Member	Groups a farmer's deliveries and their running total together, and produces their statement section.
Grade	A, B, C, REJECT are a fixed, closed set of values, each carrying its own multiplier. An enum fits this exactly.
SeasonReport	Owns the season-wide totals and the Word document generation, separate from any one member's data.
PaymentService	The five-step calculation is used by more than one part of the program. Pulling it into one place avoids repeating the formula.

Candidates that were rejected

- **Slip.** The scenario mentions paper slips, but a slip is just the old, physical version of a Delivery. Making a separate Slip class would duplicate the same fields for no extra behaviour.
- **Treasurer.** The program has only one person operating it at a time, and that person is not modelled as data, only as whoever is typing at the console. There is no behaviour or state a Treasurer class would add.
- **Cooperative.** There is only ever one cooperative in this system, REKOLT itself. A class for it would be a singleton with no real behaviour, since SeasonReport already holds the season-wide data it would represent.

4. UML class diagram

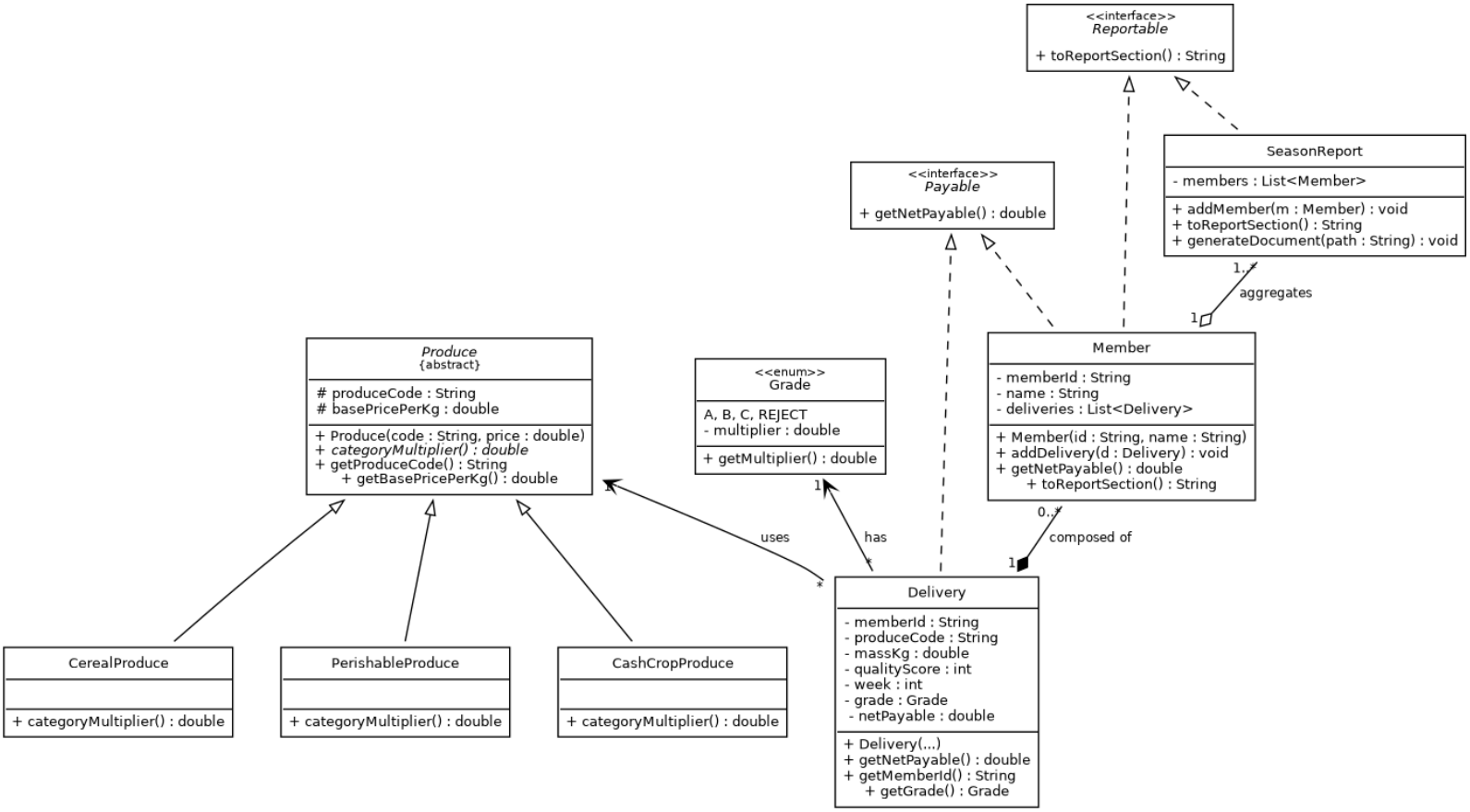
Solid arrows with an empty triangle show inheritance.

Solid arrows with an open arrowhead show association.

A filled diamond shows composition (the part cannot outlive the whole).

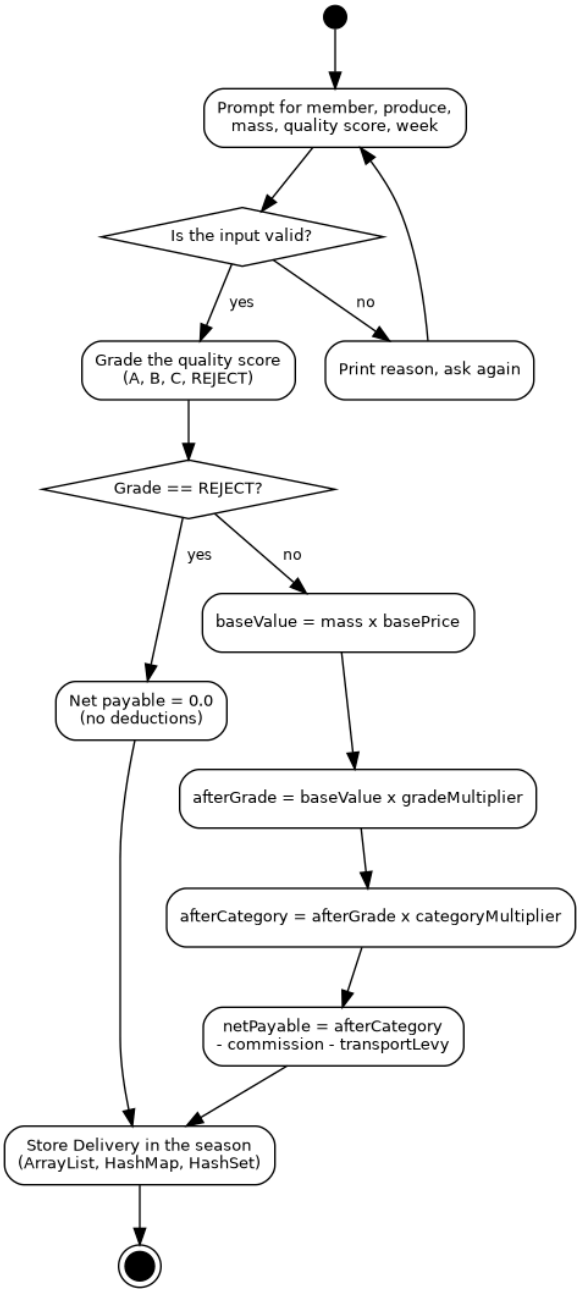
An empty diamond shows aggregation (the part can exist without the whole).

Dashed arrows with an empty triangle show interface realisation.



5. Activity diagram: recording a delivery

This traces one full pass through recording a delivery, including the input validation loop and the REJECT branch, which skips straight to a net payable of zero with no deductions.



6. Pseudocode: net payable calculation

```
FUNCTION calculateNetPayable(produceCode, massKg, qualityScore):
    grade = gradeFor(qualityScore)

    IF grade == REJECT THEN
        RETURN 0.0                  // no value, no deductions
    END IF

    produce = lookupProduce(produceCode)
    baseValue = massKg * produce.basePricePerKg
    afterGrade = baseValue * grade.multiplier
    afterCategory = afterGrade * produce.categoryMultiplier()

    commission = afterCategory * COMMISSION_RATE          // 0.05
    transportLevy = massKg * TRANSPORT_LEVY_PER_KG        // 2.0

    netPayable = afterCategory - commission - transportLevy
    RETURN round2(netPayable)
END FUNCTION


FUNCTION gradeFor(qualityScore):
    IF qualityScore >= 85 THEN RETURN A
    ELSE IF qualityScore >= 70 THEN RETURN B
    ELSE IF qualityScore >= 50 THEN RETURN C
    ELSE RETURN REJECT
    END IF
END FUNCTION
```

7. Data dictionary

Field	Type	Unit/range	Validation rule
memberId	String	format M-0000	Must match M- followed by exactly 4 digits
memberName	String	free text	Must not be empty or whitespace only
produceCode	String	MZE, BNS, POT, TEA	Must be one of the four fixed codes, case-insensitive on entry
massKg	double	kg, 0 < mass <= 5000	Must be a number, strictly greater than 0, at most 5000
qualityScore	int	0 to 100	Must be a whole number in this inclusive range
week	int	1 to 20	Must be a whole number in this inclusive range
grade	Grade (enum)	A, B, C, REJECT	Derived from qualityScore, never entered directly
netPayable	double	MUR, >= 0	Calculated, never entered directly; rounded only for display

8. Defence

1. Why an abstract Produce class instead of an if/else on produce code everywhere

The category multiplier (cereal, perishable, cash crop) is really a difference in behaviour, not just a different number sitting in a table. Putting it on an abstract method means each subclass owns its own rule, and adding a fifth produce category later only means adding one new subclass, not hunting down every if/else in the codebase that mentions produce codes. The alternative, a big switch statement everywhere the category is needed, was rejected because it spreads the same knowledge across many places instead of one.

2. Why Member owns its deliveries by composition, not aggregation

A Delivery only exists because it belongs to a member's season record. It is never shared between two members and never meaningfully exists outside of one. That is what composition means: the part's lifecycle is tied to the whole. This was chosen over aggregation, which would suggest a Delivery could be handed off between members, which does not make sense for this problem.

3. Why a REJECT grade returns early instead of running the full five steps with zero multipliers

Running REJECT through the full five-step formula (with a 0.00 grade multiplier) technically produces a base value of zero, but the commission and transport levy steps would still subtract from that zero, giving a negative result. This was actually found as a real bug while testing Objective 2. Returning 0.0 immediately once a delivery is graded REJECT was chosen over trying to patch the formula to tolerate zero, because it matches the rule directly: a REJECT delivery pays nothing and is charged nothing, full sto